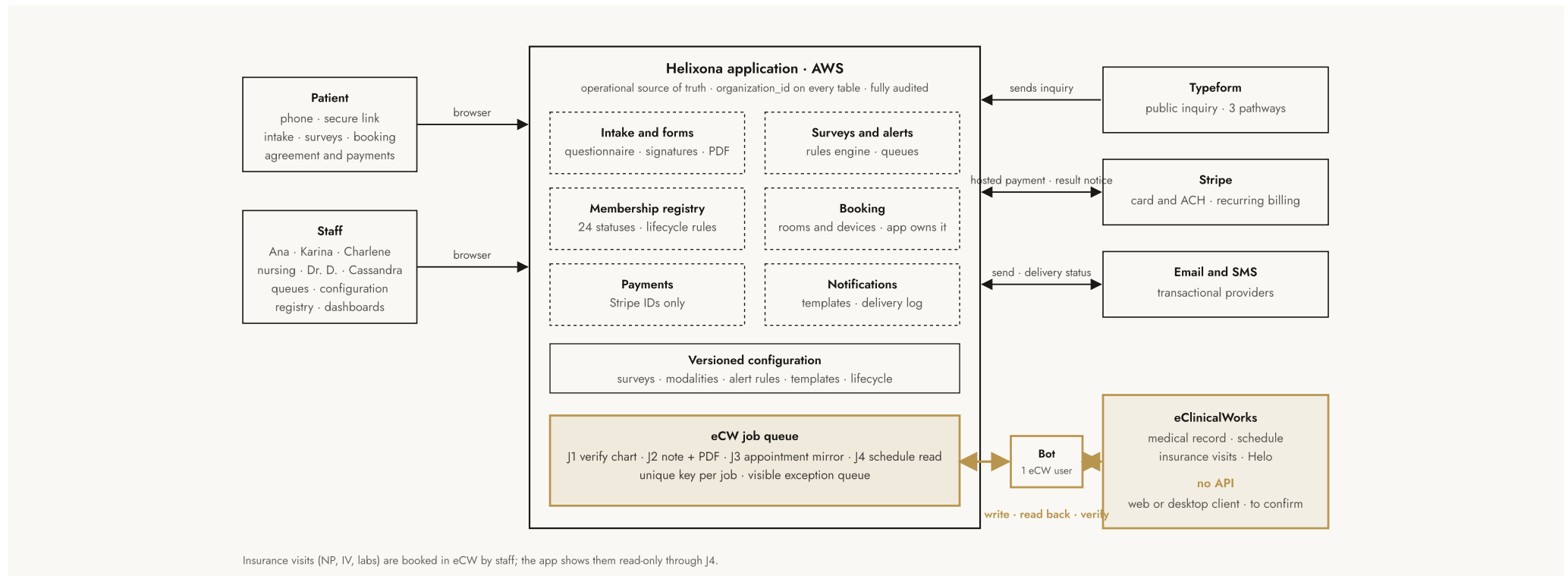


How the solution fits together

Two views: the overall architecture (what talks to what) and the eCW connector in detail (how a single bot keeps the medical record in sync without an API).

1 · High-level solution

The Helixona application is the operational layer. eClinicalWorks remains the medical record and the calendar for insurance visits. Stripe holds cards and bank accounts; the app stores only references. Everything written into eCW goes through the connector, never through an API.



Data flow of the MVP. The only path into eCW is the gold connector: it reads and writes with a dedicated user, and every write is read back before it counts as done.

WHO OWNS WHAT

App: intake, surveys, registry, modality bookings, payment references, messages. eCW: medical record and insurance appointments. Stripe: payment data.

INPUTS

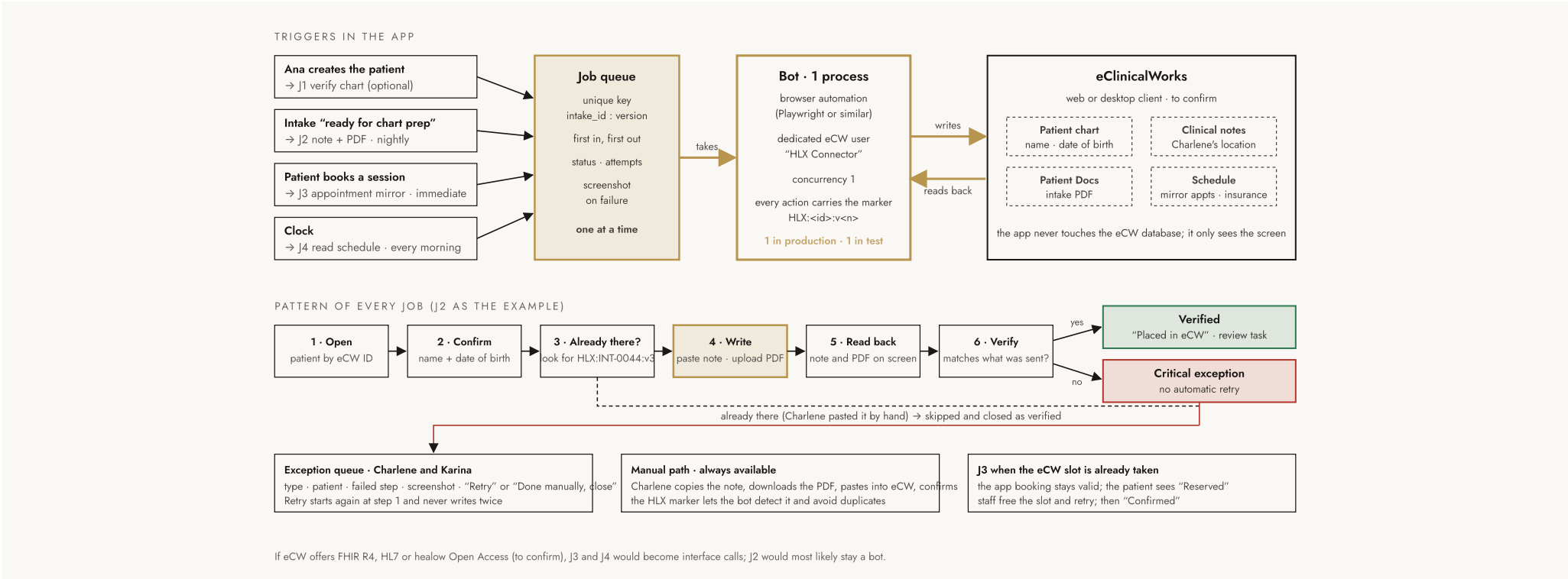
Typeform creates the inquiry. The advisor creates the patient and sends the invitation; nobody self-enrolls.

OUTPUTS

Note and PDF in eCW, mirror appointment in eCW, Stripe receipts, email and SMS with minimal personal information.

2 · The eCW connector: one bot, four jobs

A single process takes jobs from the queue, one at a time, with a dedicated eCW user. Every job type follows the same pattern: open, check it is not already done, write, read back, verify. If verification fails after a write, it stops and hands the job to a person; it never retries on its own.



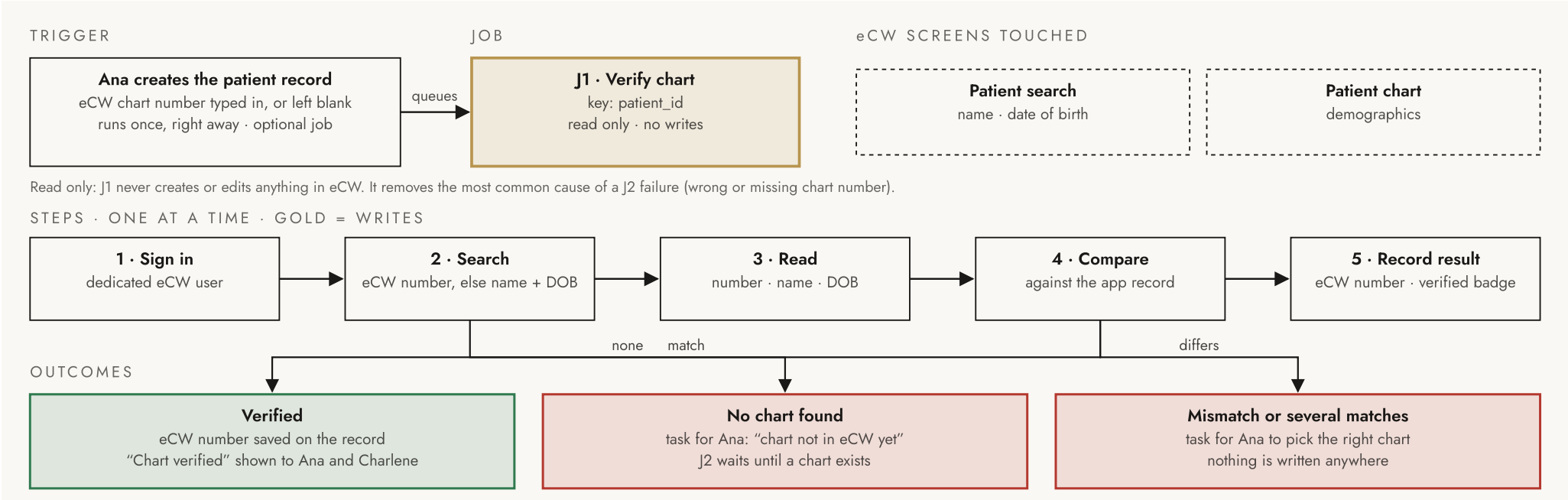
One bot, concurrency one. The four job types share the same cycle of open, check, write, read back and verify; a failed verification after a write always ends up with a person.

— eCW connector (bot) — data flow --- module inside the app

3 · Each job in detail

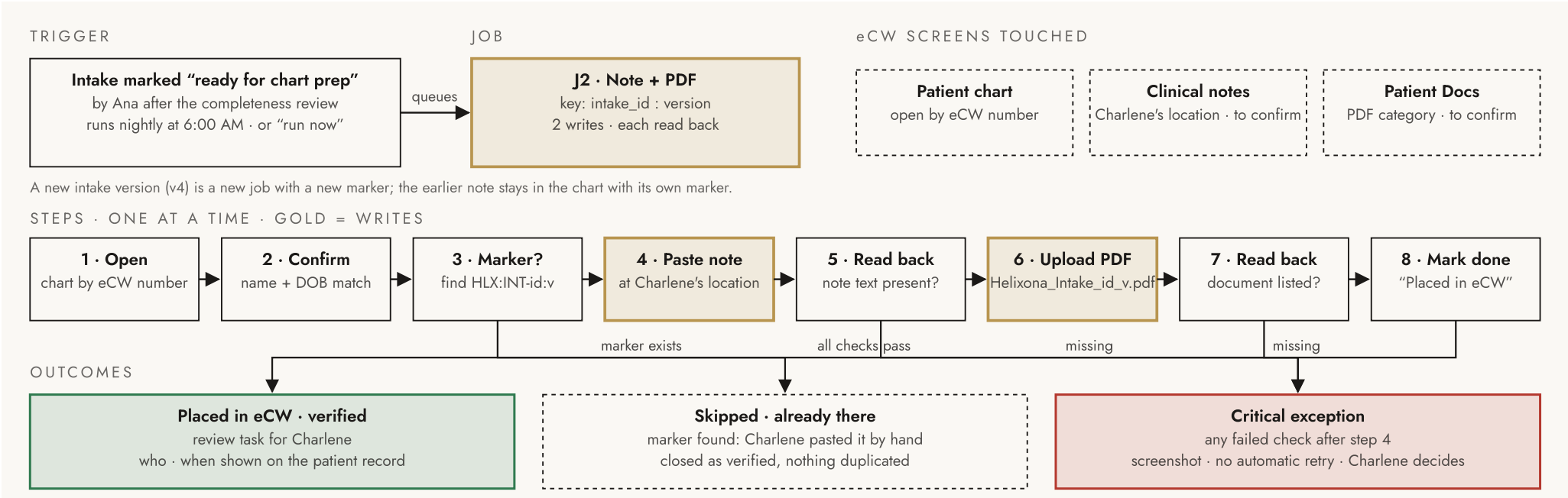
Same reading order for all four: what triggers the job, what it is, which eCW screens it touches, the steps one at a time (gold steps write), and every possible outcome with who picks it up. Only J2 and J3 write; J1 and J4 only read.

J1 · Verify chart



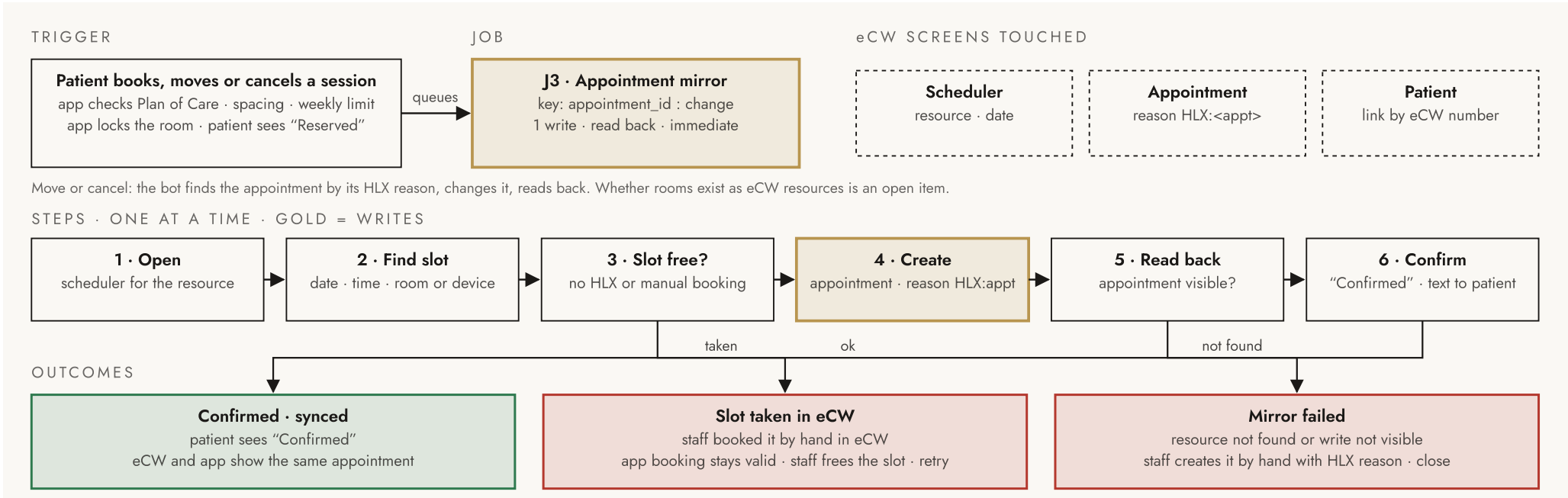
Optional, read only. Runs once when Ana creates the record so later jobs start from a verified eCW number.

J2 · Intake note + PDF



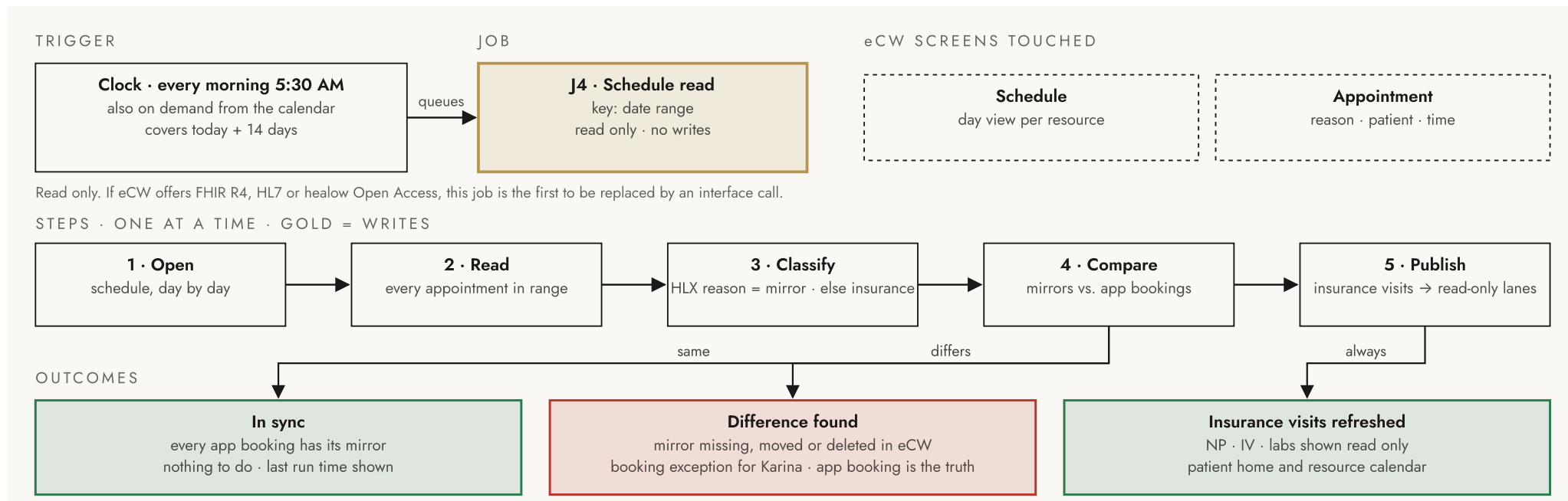
Two writes, each read back. Anything that fails after the first write stops the job and lands in Charlene’s exception queue; the bot never retries a write on its own.

J3 · Appointment mirror



The app is the real booking. eCW gets a copy with the HLX reason. The patient sees "Reserved" the moment they book and "Confirmed" only after the read-back.

J4 · Schedule read



Read only reconciliation. It is how insurance visits reach the app and how a mirror moved by hand in eCW becomes a visible exception instead of a silent mismatch.

Job table

JOB	WHEN	WHAT IT DOES IN ECW	HOW IT VERIFIES	IF IT FAILS
J1 · Chart	When the patient is created (optional)	Finds the chart, reads the eCW number, compares name and date of birth	Exact match	Task for Ana: chart not found or details differ
J2 · Note + PDF	Nightly, after “ready for chart prep”	Pastes the note at Charlene’s location, uploads the PDF to Patient Docs	Reads note and document back, looks for the HLX marker	Critical exception, no automatic retry
J3 · Appointment mirror	On booking, cancellation or reschedule	Creates the appointment with reason HLX:<appt> on the room or device	Reads the schedule back and finds the appointment	Slot taken: exception; the app booking stays valid
J4 · Schedule	Every morning and on demand	Read only: insurance visits and mirror appointments	Compares against the app’s bookings	Differences go to the booking exception queue

Sizing for the 3 to 5 patient pilot: one bot process, one eCW user in production and one in test, tens of operations a day. More clinics later means more processes running the same code, with one eCW user per clinic.